



Санкт-Петербургский
государственный университет

Exploding Kittens

Кочанов Андрей Иванович
25.543-мм

Задачи

Игра должна:

- обрабатывать ходы (одиночные карты, комбинации из 2,3,5 карт);
- поддерживать спец-карты: Attack, Skip, Favor, Shuffle, See the Future, Nope, Defuse;
- реализовывать логику взрыва и обезвреживания;
- вести историю партий и рейтинг игроков;
- иметь консольный и графический интерфейсы

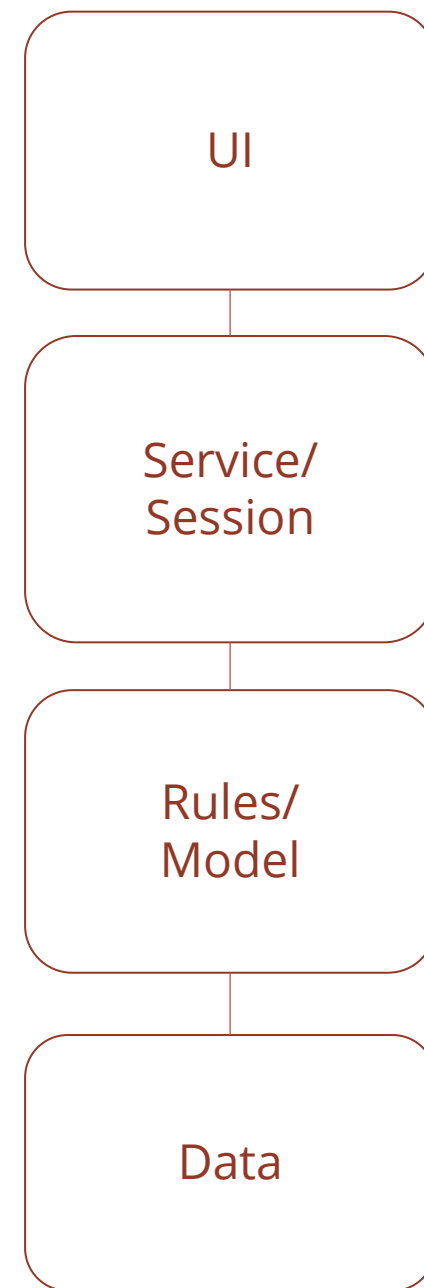
Архитектурные слои

UI – графический и консольный интерфейсы (абстракция GameManager)

Service / Session – управляет партией, статистикой, историей, реестром игроков.

Model/Rules – сущности Game, Player, DiscardPile, Deck, Card, Action; система правил RuleSet.

Data – сохранение в БД.



Модели предметной области

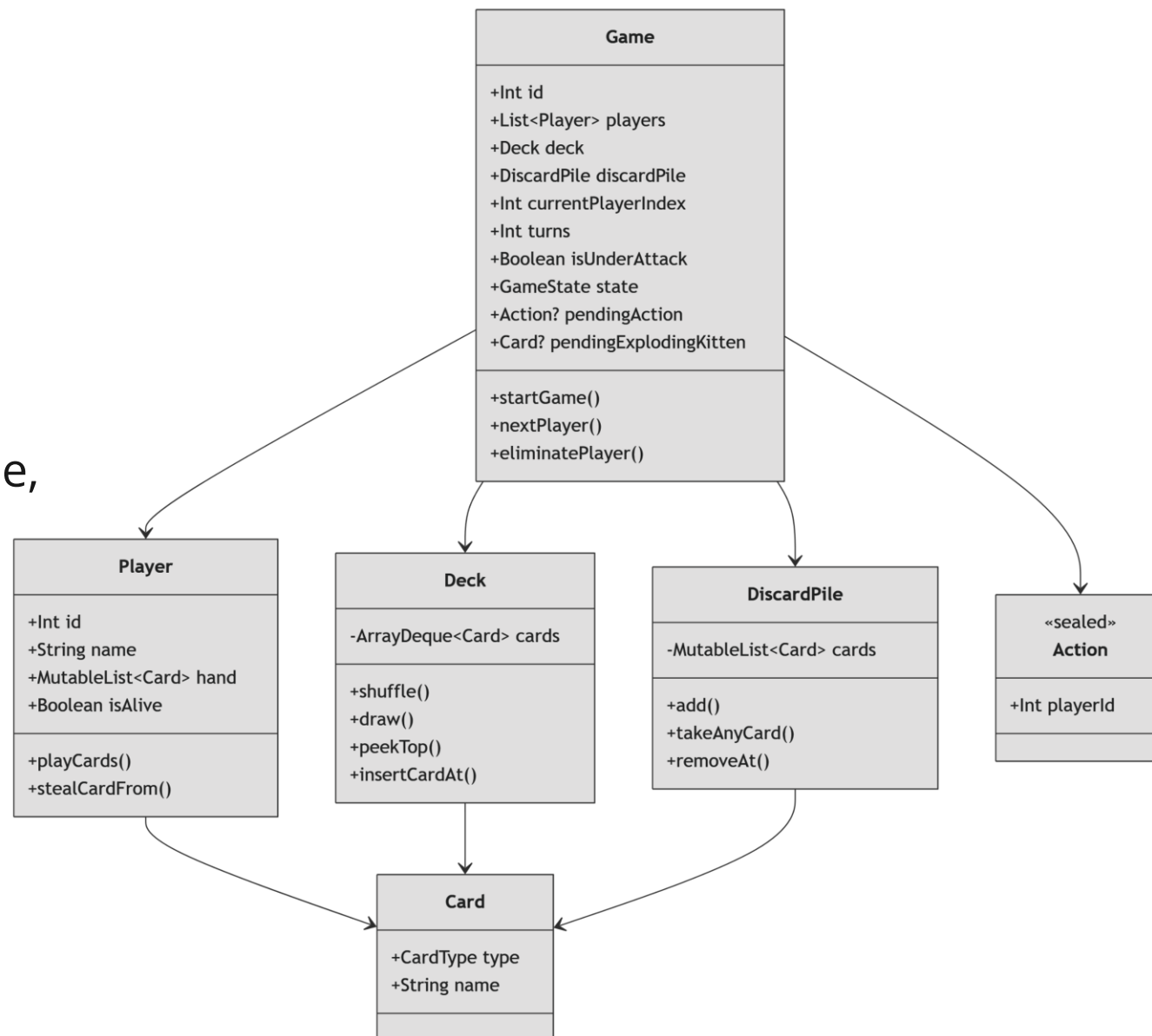
Game – состояние партии: игроки, колода, сброс, очередь хода, счётчики атаки

Player – рука, жив/мёртв

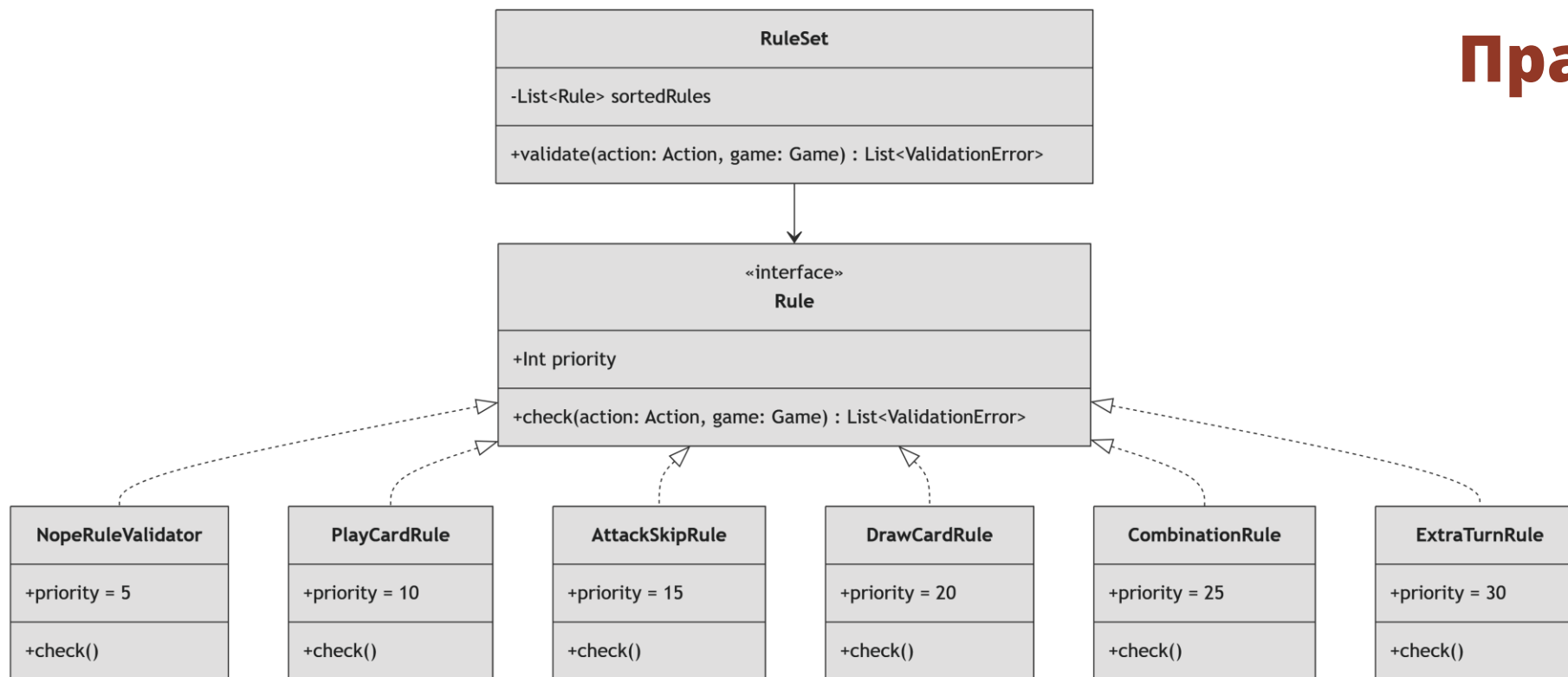
Deck – колода, методы draw, peek, insert

DiscardPile – сброс

Action – типобезопасное представление ходов (sealed class :PlayCards, Draw, UseNope, DefuseInsert, Resolve).



Правила игры



NopeRuleValidator – можно ли сыграть Nope.

PlayCardRule – нельзя играть взрыв, Defuse/Nope только в комбинации.

AttackSkipRule – запрет Attack+Skip вместе

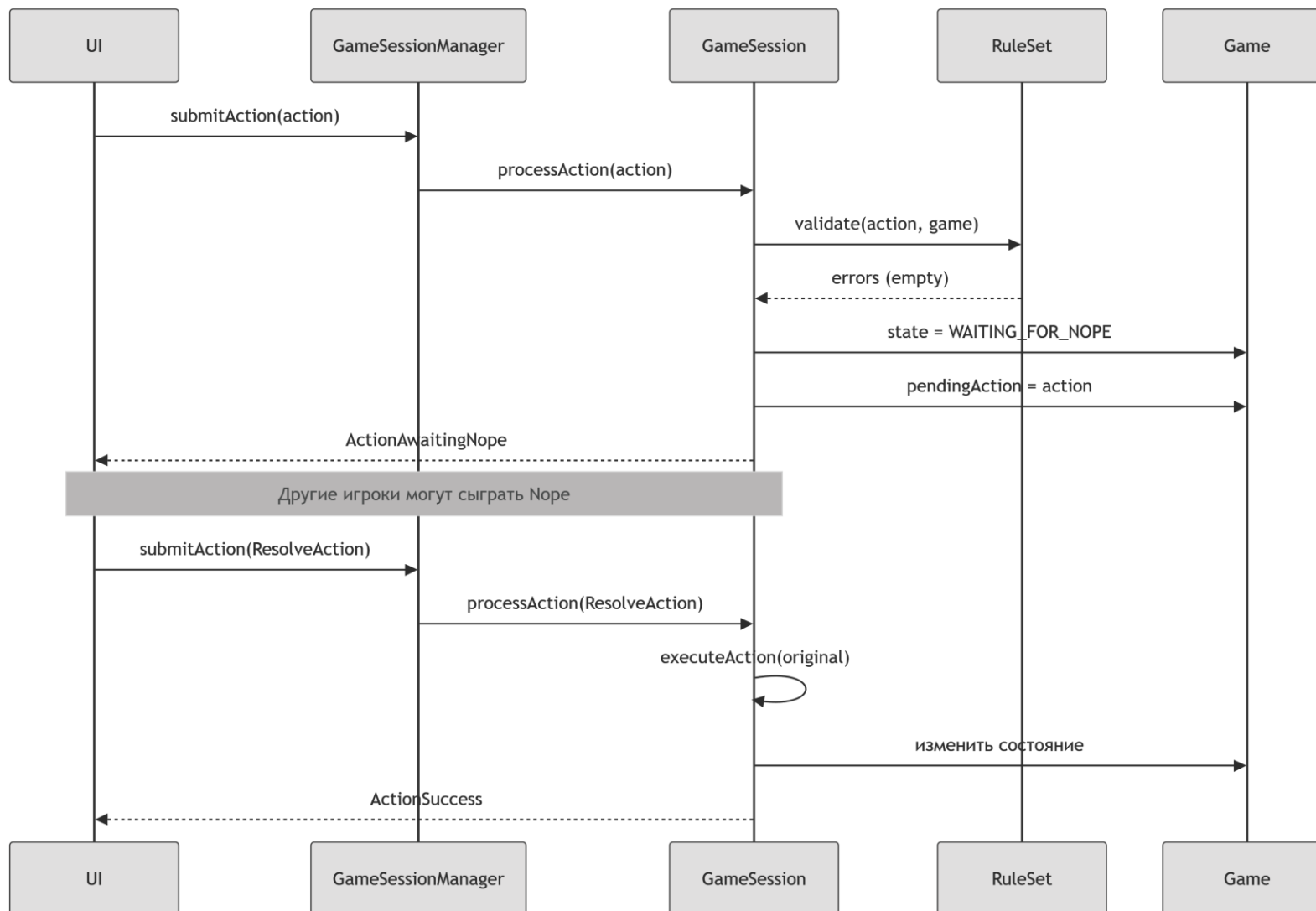
DrawCardRule – разрешено брать только в нужной фазе.

CombinationRule – валидация комбинаций.

ExtraTurnRule – обработка дополнительных ходов.

Движок игры (обработка действий)

UI отправл
GameSessic
Валидация
Если дейсте
Игроки мог
При Resolve
Исполнени
Так поддер:



ает ход).

Комбинации карт

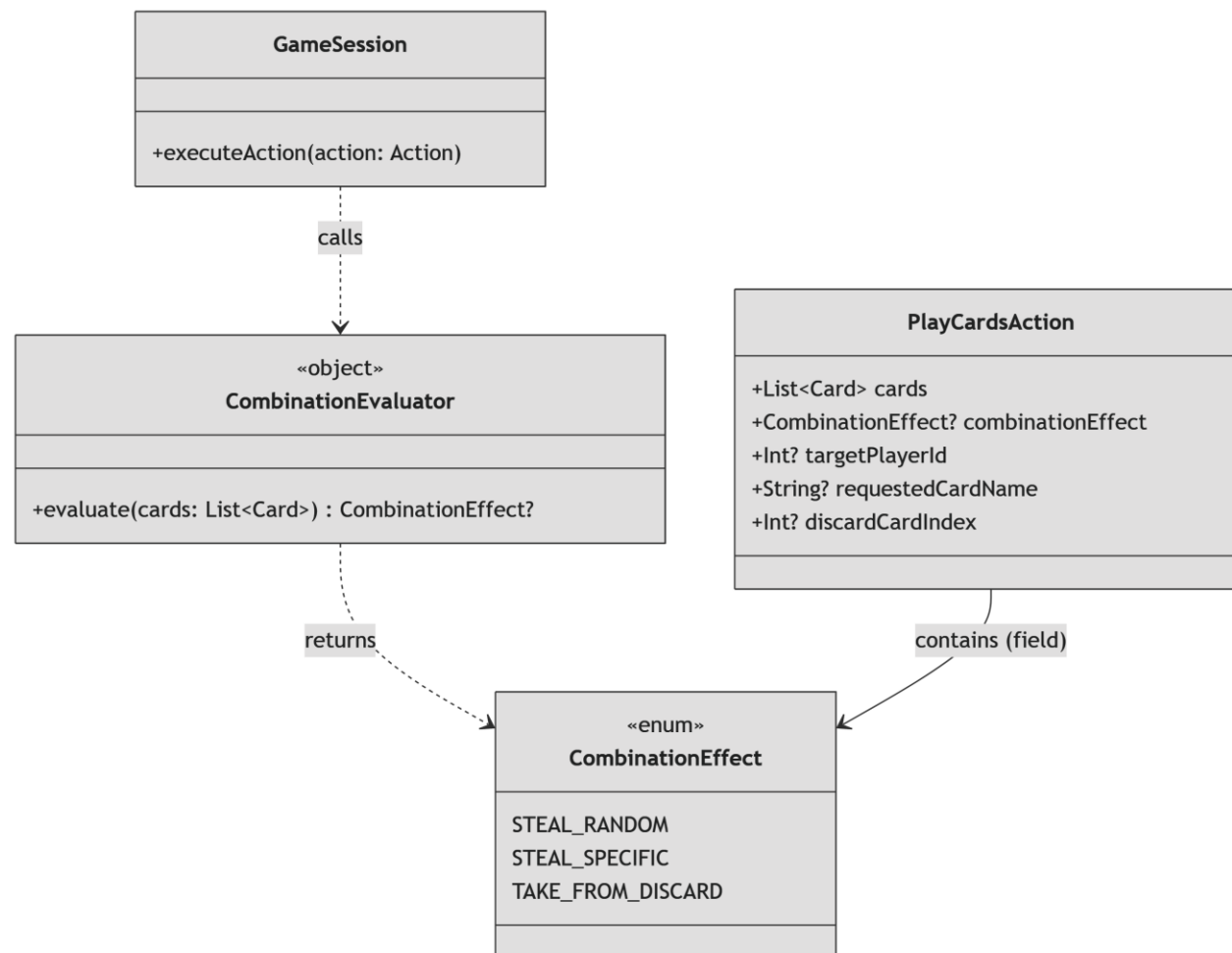
CombinationEvaluator – чистая функция:

2 одинаковые → STEAL_RANDOM (кража случайной карты).

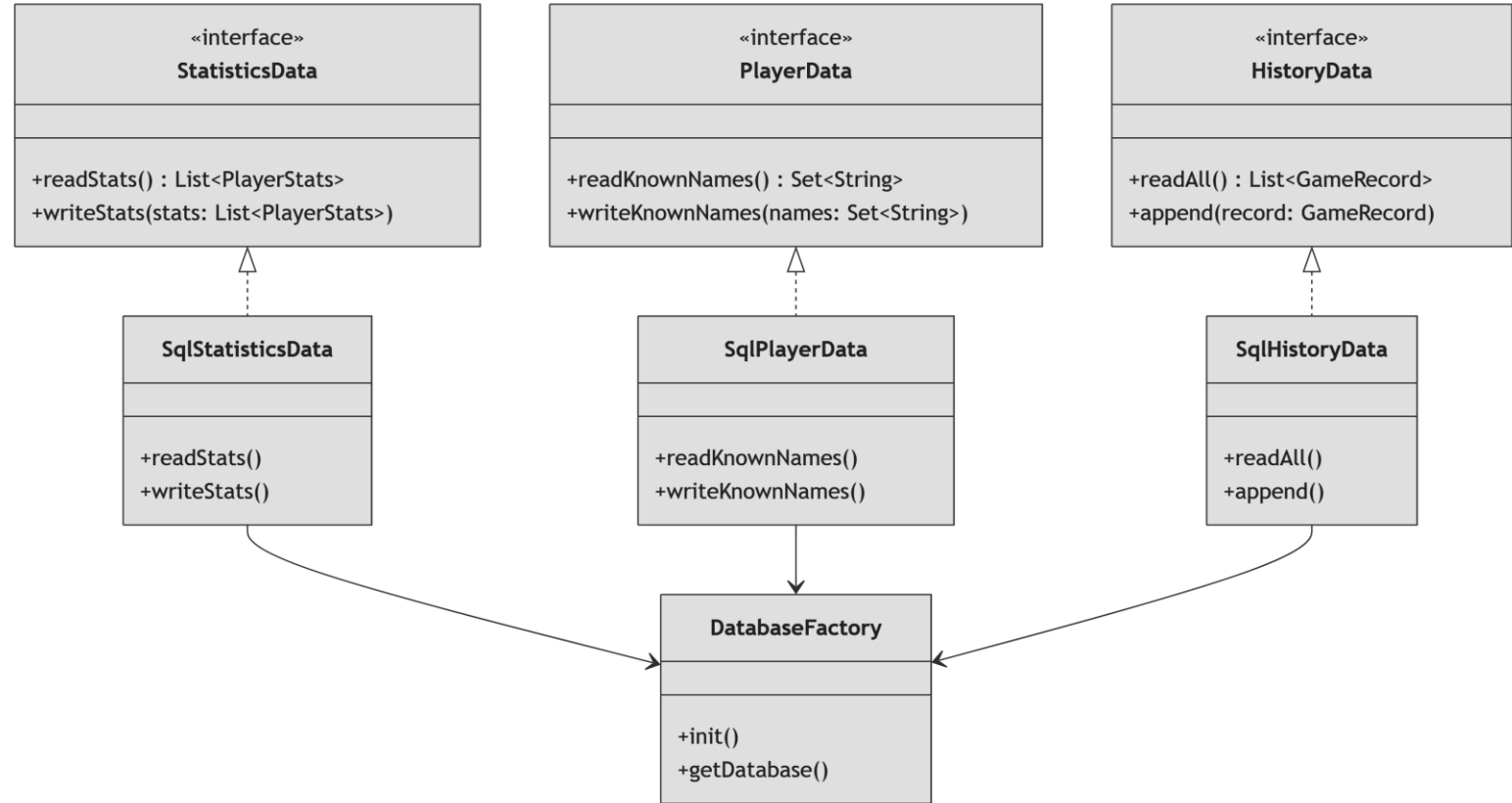
3 одинаковые → STEAL_SPECIFIC (выбор карты у другого).

5 разных кошкокарт → TAKE_FROM_DISCARD (взять любую из сброса).

Эффекты обрабатываются
в executeAction для PlayCardsAction.



Хранение данных



StatisticsData – победы/поражения.

PlayerData – список известных имён.

HistoryData – записи завершённых партий.

Реализации через Exposed
(**SqlStatisticsData**, **SqlPlayerData**, **SqlHistoryData**).

DatabaseFactory инициализирует SQLite и
создаёт таблицы.

Сервисы

(**StatisticsService**, **HistoryService**, **PlayerRegistry**)
работают только с интерфейсами – легко
заменить на JSON.

Пользовательские интерфейсы

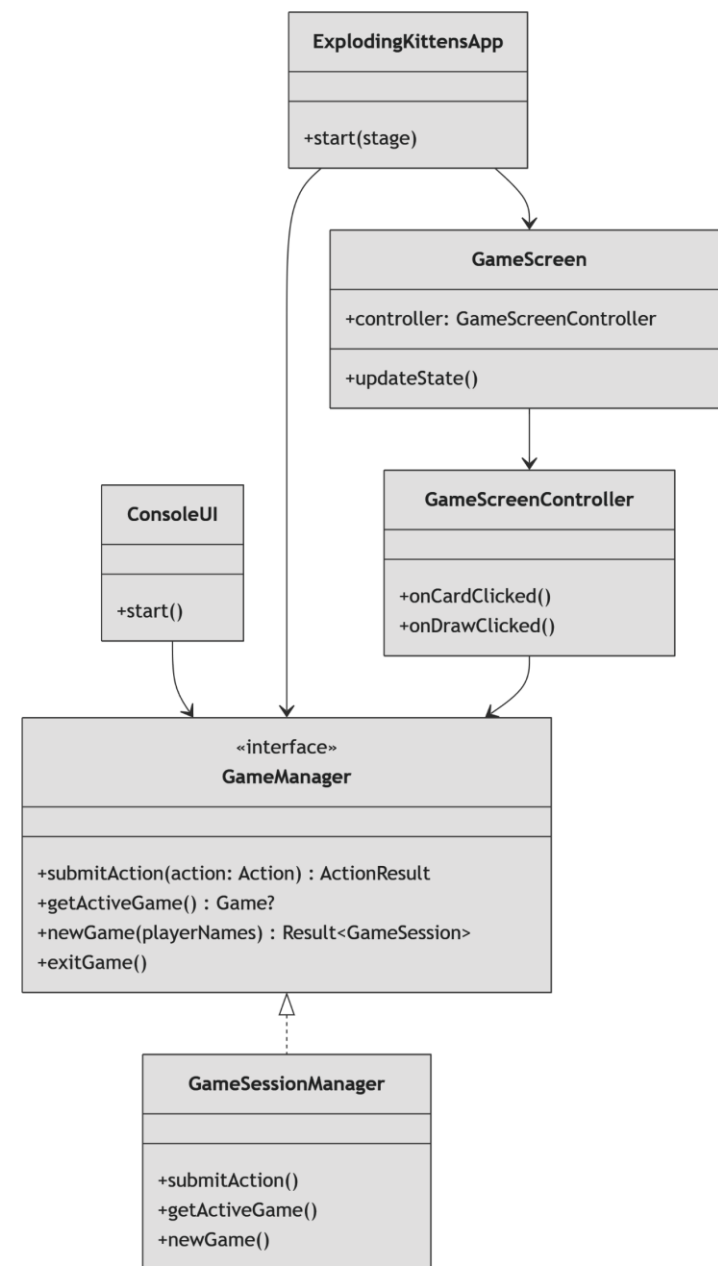
GameManager – единый интерфейс для UI.

CLI: цикл ввода команд, отображение доски текстом.

GUI: JavaFX,
классы MainMenuScreen, GameScreen, HandView, CardButton,
NopeDialog.

Оба используют submitAction, getActiveGame, newGame.

Смена UI не требует изменений в логике игры.



GUI

Select players (minimum 2)

▼

Player 2

▼

Player 3

▼

Player 4

▼

Player 5

▼

Start Game

Back

EXPLODING KITTENS

Start New Game

Exit

Current player: SpiderMen | Turns: 1

Main Menu

Your hand:

TACOCAT

SEETHEFUTURE

BEARDCAT

FAVOR

CATBUTT

FAVOR

SKIP

DEFUSE

Select Combo

Play Combo

Draw

Преимущества архитектуры

Многослойность – реализация не зависит от UI и БД.

Разделение ответственности – каждый класс решает одну задачу.

Тестируемость – правила и комбинации карт проверяются изолированно.

Гибкость через абстракции – новые режимы, правила, типы ходов добавляются без правки существующего кода.

Выполнение SOLID

- S** – Game, Rule, Action, сервисы – одна причина изменения.
- O** – новый наследник Action или Rule – открыто для расширения.
- L** – любая реализация Rule подставляется в RuleSet.
- I** – узкие интерфейсы (Rule, GameManager, *Data).
- D** – UI зависит от GameManager (абстракция), сервисы – от *Data.

Спасибо за внимание